Functional Programming Practice Variant 3

IT-2511 | Support tickets | 4 tasks | 60 minutes | 40 points

Complete STUDENT CODE and WRITTEN ANSWER Q1-Q4 in the Starter file. Total: 32 points for code and 8 for explanations. Write 2-4 sentences in English per question; code or output alone is insufficient. Answers are graded separately from code.

Timing per task, including the answer: 10 + 10 + 12 + 20 minutes; reserve 8 minutes for reading, checks and saving.

Task 1 Count accepted records

6 points total | Code 4 | Written explanation 2

Count valid records whose Priority is Medium or High and whose Age is between minAge and maxAge inclusive. Ignore Low records. Ignore invalid lines and continue processing. Return 0 for empty input or no matches. Assume all numeric parameters are in [1, 10] and minAge <= maxAge.

Each line has the format Priority;Age. The provided TryReadLine helper is correct: it accepts Low, Medium, High and an integer from 1 to 10 inclusive, trims both fields, and rejects malformed or null lines. The input sequence itself is non-null.

Correct exactly two logical errors in the method below. Keep the signature and foreach loop.

```
public static int CountAcceptedTickets(IEnumerable<string> lines, int minAge, int maxAge)
{
    int count = 0;
    foreach (string line in lines)
    {
        if (!TryReadLine(line, out string kind, out int value))
            continue;

        if (kind == "Medium" ||
            kind == "High" && value > minAge && value <= maxAge)
            count++;
    }
    return count;
}
```

Example input: "Medium;2"; "Medium;6"; "High;3"; "High;5"; "Low;4"; "Medium;4". With minAge = 3, maxAge = 5, expected result: 3.

In DemoTask1, add two Check(label, expectedInt, actualInt) calls. Each input must expose a different bug: fail before the fixes and pass afterwards. Do not reuse the whole provided example.

Required explanation Q1

Why must the Age range apply to both Medium and High? With the range 3..5, give one line exposing the grouping error and one line exposing the inclusive boundary error.

CRITERION	POINTS
FIRST LOGICAL ERROR CORRECTED	1
SECOND LOGICAL ERROR CORRECTED	1
TWO DIAGNOSTIC CHECKS EXPOSING DIFFERENT ERRORS	1 + 1
WRITTEN EXPLANATION Q1	2

Task 2 Higher-order scoring

8 points total | Code 6 | Written explanation 2

Provided model

```
public enum TicketPriority { Low, Medium, High }
public record SupportTicket(int Id, string Title, int Age, bool IsOpen, TicketPriority Priority);
```

Assume valid non-null records: unique positive IDs, non-null text, Age from 1 to 10, and only declared enum values. All supplied function parameters are non-null. Do not add validation for these assumptions.

Methods to implement

```
public static int Evaluate(SupportTicket item, Func<SupportTicket, int> scoreRule, Func<int, int> adjustment)
public static int BaseScore(SupportTicket item)
```

1. Evaluate must calculate scoreRule(item) exactly once, pass that score to adjustment exactly once, and return the adjusted result.

2. BaseScore must use a switch expression on Priority. Select the base points from the table, then add Age * 2.

PRIORITY	BASE POINTS
LOW	4
MEDIUM	9
HIGH	15

3. In DemoTask2, make the two calls shown in the Starter file and print both returned values. Both methods and all supplied functions must be pure: no console or file operations inside them, no input mutation, and no external-state changes.

Sample object: new SupportTicket(99, "Sample", 3, true, TicketPriority.High). BaseScore(sample) = 21. Expected Evaluate results: 19 and 42.

Required explanation Q2

Suppose scoreRule(item) returns 18. What does Evaluate(item, scoreRule, x => x - 4) return? Explain the order of the two function calls and whether the method stays pure if adjustment changes global state.

CRITERION	POINTS
SCORE RULE FOLLOWED BY ADJUSTMENT	2
CORRECT BASE MAPPING USING A SWITCH EXPRESSION	2
PURE METHODS AND SUPPLIED FUNCTIONS	1
TWO CORRECT DEMO CALLS AND PRINTED RESULTS	1
WRITTEN EXPLANATION Q2	2

Task 3 Filtered adjacent sums

10 points total | Code 8 | Written explanation 2

For each pair of neighbouring values, calculate their sum. Return that result only if it is greater than or equal to minSum.

```
public static IEnumerable<int> AdjacentSums(IEnumerable<int> values, int minSum)
```

Requirements

Use yield return. Traverse values once per enumeration and keep a fixed number of local variables. Do not use LINQ, indexing, Count(), repeated traversal, a temporary array or list, or shared mutable fields.

The sequence must be lazy: calling the method must not consume input values. Empty input and a single input value must both produce an empty result. Preserve the original order of neighbouring pairs. A skipped pair must not change which values are neighbours in the source sequence.

Assume values is non-null and every integer is from -1000 to 1000. Do not change the input or print inside the iterator. Demonstration calls are already provided; you do not need to write them.

INPUT / PARAMETER	EXPECTED RESULT
4, 1, 6, 2, 8; MINSUM=7	7, 8, 10
2, 5, 1; MINSUM=5	7, 6
-3, 5, -1, 4; MINSUM=3	4, 3
7; MINSUM=5	<empty>
<EMPTY>; MINSUM=5	<empty>

Required explanation Q3

If the result of a neighbouring pair fails the filter, which source value must become previous for the next pair? Explain why updating previous only when a result is returned gives wrong pairs.

CRITERION	POINTS
CORRECT PAIR RESULTS, FILTER AND ORIGINAL ORDER	3
LAZY GENERATION WITH YIELD RETURN	2
ONE TRAVERSAL AND FIXED EXTRA STORAGE	1
EMPTY INPUT RETURNS AN EMPTY SEQUENCE	1
SINGLE VALUE RETURNS AN EMPTY SEQUENCE	1
WRITTEN EXPLANATION Q3	2

Task 4 Build a ranked report

16 points total | Code 14 | Written explanation 2

Use the provided model and sample data. This task does not depend on your solutions to Tasks 2 or 3. Implement the method using LINQ for filtering, projection, ordering and limiting.

```
public static string[] BuildReport(IEnumerable<SupportTicket> items, Func<SupportTicket, int> rule, int minAge, int count)
```

1. Keep only records where IsOpen is true.

2. Keep only records where Age is greater than or equal to minAge.

3. Keep only records where Priority is not Low.

4. Calculate scores with the supplied rule. Retain each computed score for sorting and output; do not score the same eligible record more than once.

5. Sort by score in descending order. For equal scores, sort by Id in descending order.

6. Return at most count strings in the exact format "Id: Title [score]". For count = 0 or no matches, return an empty array. If fewer records qualify, return all of them.

Assume valid records as in Task 2, a non-null input, a non-null pure rule, minAge in [1, 10], and count >= 0. No input mutation or console output is allowed inside BuildReport.

ID	TITLE	AGE	ISOPEN	PRIORITY
9	Login	4	true	High
4	Payment	4	true	High
2	Printer	4	true	Low
1	Network	3	true	High
7	Account	4	false	High
6	Email	6	true	Medium

With minAge = 4, count = 3 and this independent rule:

```
x => x.Age * 3 + (x.Priority == TicketPriority.High ? 4 : 0)
```

Expected strings, in order:

6: Email [18]

9: Login [16]

4: Payment [16]

Required explanation Q4

Why must the Priority filter be applied before Take(count)? Why is ThenByDescending used for equal scores instead of a second OrderByDescending?

CRITERION	POINTS
STATUS FILTER	1.5
INCLUSIVE NUMERIC LIMIT	1.5
ADDITIONAL MODEL FILTER	1
SUPPLIED RULE AND REUSE OF SCORES	2 + 1
PRIMARY ORDERING AND TIE ORDERING	2 + 2
COUNT LIMIT; FORMAT/ARRAY; ZERO COUNT; NO MATCHES	1 + 1 + 0.5 + 0.5
WRITTEN EXPLANATION Q4	2